



MemSient

Product Requirements Document (PRD) v1.0

The Sentient Memory Platform for AI Agents

"Memory is not storage — it's the foundation of intelligence."



Table of Contents

1. [Executive Summary](#)
 2. [Vision, Mission & Values](#)
 3. [Market Analysis](#)
 4. [Target Customers & Use Cases](#)
 5. [Product Overview](#)
 6. [Core Platform Features](#)
 7. [Technical Architecture](#)
 8. [Platform Components](#)
 9. [Integration Methods](#)
 10. [Developer Experience](#)
 11. [Infrastructure & Deployment](#)
 12. [Security, Privacy & Compliance](#)
 13. [Business Model & Pricing](#)
 14. [Go-to-Market Strategy](#)
 15. [Success Metrics & KPIs](#)
 16. [Product Roadmap](#)
 17. [Competitive Analysis](#)
 18. [Risk Assessment & Mitigation](#)
 19. [Team & Resources](#)
 20. [Appendices](#)
-

1. Executive Summary

1.1 The Problem

The AI agent revolution is here. Coding assistants like Cursor and Windsurf, research agents, legal AI, financial advisors, and countless other agentic applications are transforming every industry. But they all share a critical weakness:

AI agents have no memory.

Current approaches fail spectacularly at scale:

Approach	Day 1	Day 100	Day 365
Context Stuffing	Works	Token overflow	Impossible
Vector Databases	Works	Contradictions accumulate	Knowledge garbage
RAG Systems	Works	No temporal awareness	Outdated info retrieved
Key-Value Stores	Works	No relationships	Fragmented knowledge

The result? AI agents that:

- ❌ Forget critical information
- ❌ Retrieve outdated facts as current
- ❌ Can't resolve contradictions
- ❌ Lose context across sessions
- ❌ Fail to learn and improve over time

This is the \$50B+ opportunity that MemSient captures.

1.2 The Solution

MemSient is the world's first **Sentient Memory Platform** — a living, self-evolving memory and context layer for AI agents that thinks like a human brain.

Unlike static storage systems, MemSient:

-  **Forgets** irrelevant information naturally (Memory Decay)
-  **Consolidates** related knowledge automatically (Memory Consolidation)
-  **Prioritizes** important information (Importance Scoring with PageRank)
-  **Tracks time** with temporal validity ("CEO in 2015" vs "CEO now")
-  **Resolves conflicts** between old and new information
-  **Learns continuously** from every interaction
-  **Evolves** its knowledge graph over time

1.3 The Platform

MemSient provides:

1. **Cloud Platform** — Fully managed, globally distributed memory infrastructure
2. **Edge Runtime** — WASM-compiled core for on-device deployment
3. **MCP Integration** — Native Model Context Protocol support for AI frameworks
4. **Universal SDK** — Python, TypeScript, Rust, Go clients
5. **REST/GraphQL API** — Standard APIs for any application
6. **Enterprise Features** — SOC2, HIPAA, multi-tenancy, SSO

1.4 Key Metrics (Targets - Year 1)

Metric	Target
Platform Customers	500+ companies
API Requests/Month	1 billion+
Memory Operations/Sec	100,000+ peak
ARR	\$10M+
Query Latency (p99)	< 100ms
Uptime	99.99%

2. Vision, Mission & Values

2.1 Vision

To become the universal memory layer for artificial intelligence — enabling every AI agent to remember, learn, and evolve like a sentient being.

By 2030, MemSient will power the memory of:

- 1 million+ AI agents worldwide
- 100+ agentic platforms
- Trillions of memories stored and evolved

2.2 Mission

To give AI agents the gift of memory — transforming stateless language models into intelligent beings that learn from experience, remember what matters, and forget what doesn't.

2.3 10-Year Vision: The Path to Beneficial Superintelligence

MemSient is not just a memory platform — it's the foundational layer for the path to AGI and beyond. Our 10-phase technical roadmap:

Phase	Milestone	Status
1	Graph-Based Memory	✅ Complete
2	Self-Evolving Memory	✅ Complete
3	Skill Acquisition Engine	🔄 In Progress
4	Program Abstraction	📅 Planned
5	ARC-AGI 100%	🎯 Key Milestone

Phase	Milestone	Status
6	Meta-Cognition Layer	📅 Planned
7	World Modeling	📅 Planned
8	Collective Intelligence	📅 Planned
9	Artificial General Intelligence	📅 Long-term
10	Aligned Superintelligence	📅 Long-term

2.4 Core Values

🧠 Intelligence First

Every feature must make AI agents smarter, not just faster.

🔒 Privacy by Design

Memory is intimate. We protect it with the highest standards.

⚡ Speed of Light

Memory access must be instant. Latency kills intelligence.

🌐 Universal Compatibility

Any agent, any platform, any language. Memory should be universal.

🔬 Research-Driven

Our innovations come from neuroscience and cognitive research, not marketing.

🤝 Open Core Philosophy

The core algorithms are open. Enterprise features sustain the business.

3. Market Analysis

3.1 Total Addressable Market (TAM)

AI Agent Infrastructure Market

Segment	2024	2027 (Projected)	CAGR
AI Agent Platforms	\$5.2B	\$28.4B	76%
AI Development Tools	\$12.8B	\$45.2B	52%
Enterprise AI Infrastructure	\$31.5B	\$98.7B	46%
Total TAM	\$49.5B	\$172.3B	52%

MemSient's Serviceable Market (SAM)

Focusing on companies building or deploying AI agents:

- **SAM 2024:** \$8.2B
- **SAM 2027:** \$35.6B

Obtainable Market (SOM)

With aggressive execution:

- **Year 1:** \$10M ARR (0.12% of SAM)
- **Year 3:** \$100M ARR (0.5% of SAM)
- **Year 5:** \$500M ARR (1.4% of SAM)

3.2 Market Trends Driving Adoption

1. Explosion of AI Agents

- Cursor: 100,000+ users, \$100M ARR

- GitHub Copilot: 1.5M+ users
- ChatGPT: 200M+ users
- Enterprise AI adoption: 72% of Fortune 500

2. Context Window Limitations

Even GPT-4 Turbo (128K) cannot hold a year of agent history.

3. Regulatory Pressure

GDPR, AI Act, and industry regulations require:

- Explainable AI decisions
- Audit trails
- Data isolation
- Right to be forgotten

4. Enterprise AI Scaling

Companies moving from pilot to production need:

- Multi-tenant isolation
- Horizontal scaling
- Cost predictability
- Reliability guarantees

5. Model Context Protocol (MCP) Adoption

Anthropic's MCP is becoming the standard for AI-tool integration, creating demand for memory providers.

3.3 Competitive Landscape

Direct Competitors

Company	Approach	Weakness
Pinecone	Vector database	No graph, no evolution, no temporal
Weaviate	Vector + graph	No memory evolution, basic temporal
Chroma	Lightweight vectors	No enterprise features
Milvus	Scalable vectors	Complexity, no cognitive features
LangChain Memory	Framework memory	Not persistent, not scalable
Mem0	Memory layer	Basic features, no evolution

MemSient's Differentiation

Feature	Pinecone	Weaviate	Mem0	MemSient
Memory Decay	✗	✗	✗	✓
Consolidation	✗	✗	✗	✓
Temporal Validity	✗	Basic	✗	✓ Full
Knowledge Graph	✗	✓	✗	✓
Importance Scoring	✗	✗	✗	✓ PageRank
Conflict Resolution	✗	✗	✗	✓
Self-Evolution	✗	✗	✗	✓
Edge Deployment	✗	✗	✗	✓ WASM
MCP Native	✗	✗	✗	✓

MemSient is the only platform with cognitive memory features inspired by neuroscience.

4. Target Customers & Use Cases

4.1 Primary Customer Segments

Segment 1: AI Agent Platforms (B2B)

Companies building agentic applications

Company Type	Examples	Pain Point	MemSient Value
Coding Assistants	Cursor, Windsurf, Cody	Context across sessions	Full project memory
AI IDEs	Replit, CodeSandbox	Long-term learning	Developer preferences
Research Agents	Elicit, Consensus	Document synthesis	Cross-paper knowledge
Legal AI	Harvey, Casetext	Case history	Precedent memory
Financial AI	Bloomberg, Kensho	Market context	Temporal trends
Customer Support	Intercom, Zendesk	Customer history	Relationship memory
Sales AI	Gong, Outreach	Deal context	Conversation memory

Estimated Segment Size: 5,000+ companies globally

Target Year 1: 100 platform customers

Average Contract Value: 50K – 500K ARR

Segment 2: Enterprise AI Teams (B2B)

Companies building internal AI agents

Industry	Use Case	MemSient Value
Financial Services	Trading assistants, risk analysis	Temporal market memory
Healthcare	Clinical decision support	Patient history synthesis
Manufacturing	Process optimization agents	Operational knowledge
Retail	Personalization agents	Customer journey memory
Media	Content recommendation	Preference evolution

Estimated Segment Size: 10,000+ enterprise AI teams

Target Year 1: 300 enterprise customers

Average Contract Value: 20K – 100K ARR

Segment 3: AI Developers (B2D)

Individual developers and small teams building agents

Developer Type	Use Case	MemSient Value
Indie Hackers	Personal AI assistants	Easy memory integration
Startups	MVP agentic products	Scalable from day 1
Researchers	Experimental agents	Academic discounts
Hobbyists	Side projects	Free tier access

Estimated Segment Size: 500,000+ AI developers

Target Year 1: 10,000 developers

Revenue Model: Freemium → Paid conversion

4.2 Detailed Use Cases

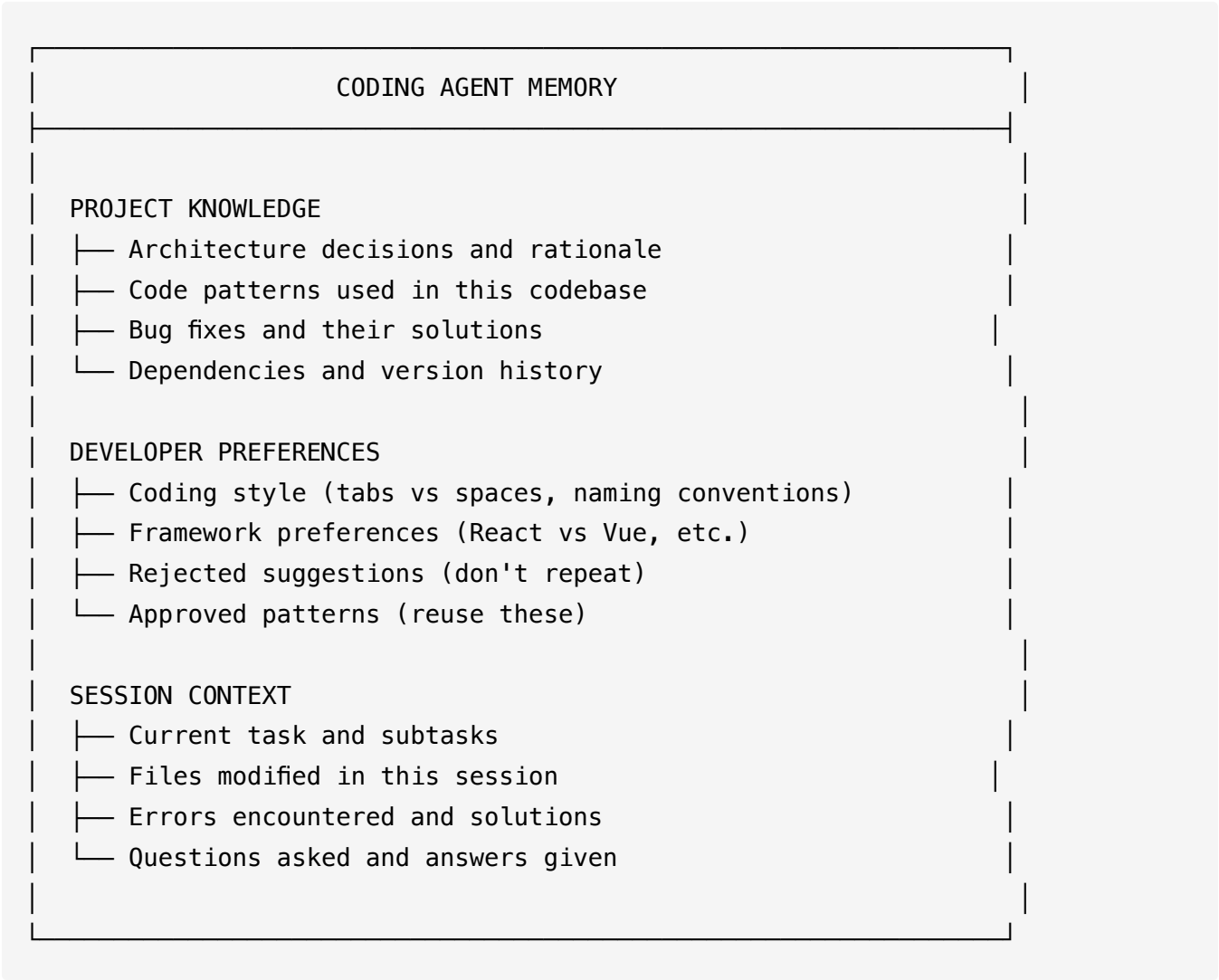
Use Case 1: Coding AI Assistants

Customer Profile: Cursor, Windsurf, Cody, Continue

Current Pain:

- Lost context when switching between files
- No memory of previous debugging sessions
- Can't learn developer preferences over time
- Repeats same suggestions that were rejected

MemSient Solution:



Value Delivered:

- 40% reduction in repeated suggestions
- 3x faster context recovery across sessions
- Personalized assistance that improves over time

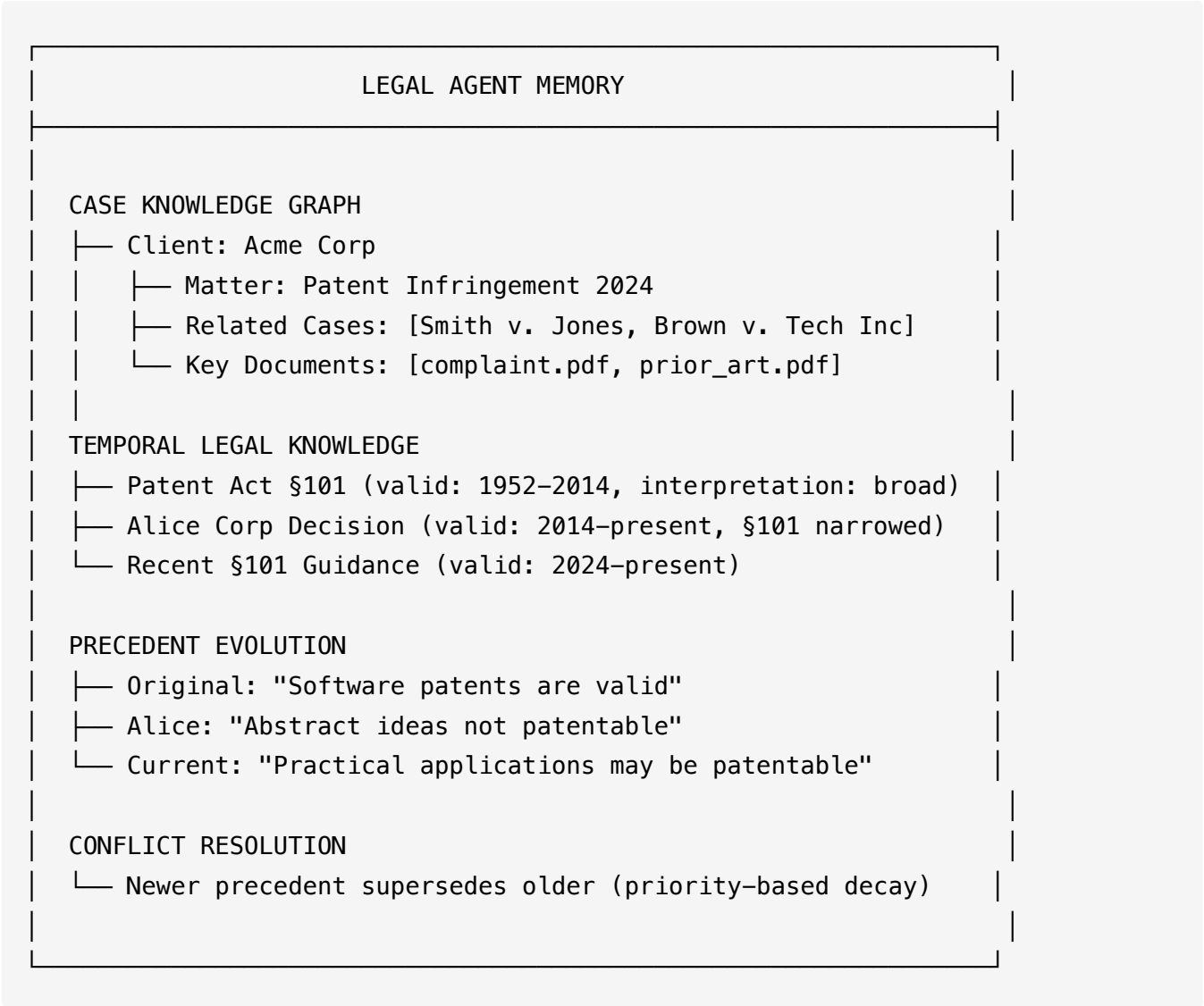
Use Case 2: Legal AI Assistants

Customer Profile: Harvey, Casetext, LexisNexis AI

Current Pain:

- Can't track precedent evolution over time
- No memory of case-specific context
- Contradictory legal interpretations
- Missing temporal validity of laws

MemSient Solution:



Value Delivered:

- Accurate temporal legal advice
- Automatic precedent tracking
- Reduced research time by 60%

Use Case 3: Research Assistants

Customer Profile: Elicit, Consensus, Semantic Scholar

Current Pain:

- Can't synthesize across hundreds of papers
- No tracking of research evolution
- Missing citation relationships
- Conflicting study results not handled

MemSient Solution:

RESEARCH AGENT MEMORY
KNOWLEDGE SYNTHESIS
└ Topic: "Transformer Architectures"
└ └ Original Transformer (Vaswani 2017)
└ └ └ BERT (Devlin 2018) → extends Transformer
└ └ └ GPT-3 (Brown 2020) → scales Transformer
└ └ └ Current SOTA: GPT-4, Claude, Gemini (2024)
CLAIM TRACKING
└ Claim: "Scaling improves reasoning"
└ └ Support: [GPT-3, PaLM, Chinchilla]
└ └ Contradict: [Hoffmann 2022 - compute scaling]
└ └ Current Consensus: "Depends on task"
METHODOLOGY MEMORY
└ Benchmark: MMLU
└ └ Created: 2020
└ └ Saturated: 2024 (90%+ achieved)
└ └ Superseded by: MMLU-Pro, ARC-AGI

Value Delivered:

- Cross-paper synthesis
- Automatic claim tracking
- Research evolution timeline

Use Case 4: Financial Trading Agents

Customer Profile: Bloomberg, Kensho, Two Sigma

Current Pain:

- Market regime changes not tracked
- No memory of pattern evolution

- Temporal data relationships lost
- Conflicting signals not resolved

MemSient Solution:

FINANCIAL AGENT MEMORY	
MARKET REGIME MEMORY	
└─ Regime: Bull Market (2020–2021)	
└─┬─ Patterns: Tech outperformance, low volatility	
└─ Regime: Bear Market (2022)	
└─┬─ Patterns: Value rotation, high volatility	
└─ Regime: Recovery (2023–present)	
└─┬─ Patterns: AI stocks, mixed signals	
TEMPORAL RELATIONSHIPS	
└─ Fed Rate: 0.25% → 5.25% (2022–2023)	
└─┬─ Impact: Growth stocks declined 40%	
└─ Inflation: 9.1% → 3.2% (2022–2024)	
└─┬─ Impact: Bond yields normalized	
SIGNAL EVOLUTION	
└─ Original: "Inverted yield curve = recession"	
└─ Update: "Delay extended due to fiscal stimulus"	
└─┬─ Current: "Soft landing increasingly likely"	

Value Delivered:

- Regime-aware trading signals
 - Temporal pattern tracking
 - Conflict resolution for mixed signals
-

5. Product Overview

5.1 Product Definition

MemSient is a **Sentient Memory Platform** that provides:

1. **Memory Storage** — Persistent, structured, temporal knowledge storage
2. **Memory Evolution** — Automatic consolidation, decay, and improvement
3. **Memory Retrieval** — Semantic search, graph traversal, context assembly
4. **Memory Integration** — MCP, SDK, API for any agentic application

5.2 Core Product Principles

Principle 1: Cognitive Memory, Not Storage

MemSient is not a database. It's a cognitive memory system that:

- Forgets irrelevant information (decay)
- Consolidates related knowledge (clustering)
- Prioritizes important facts (PageRank)
- Tracks temporal validity (when things were true)

Principle 2: Speed of Light

Memory must be instant. Our targets:

- **p50 latency:** < 50ms
- **p99 latency:** < 100ms
- **Ingestion:** 10,000+ docs/second
- **Edge latency:** < 10ms

Principle 3: Universal Compatibility

Works with any:

- **LLM:** OpenAI, Anthropic, Google, Mistral, Ollama
- **Framework:** LangChain, LlamaIndex, AutoGen, CrewAI
- **Language:** Python, TypeScript, Rust, Go, Java
- **Deployment:** Cloud, on-prem, edge, hybrid

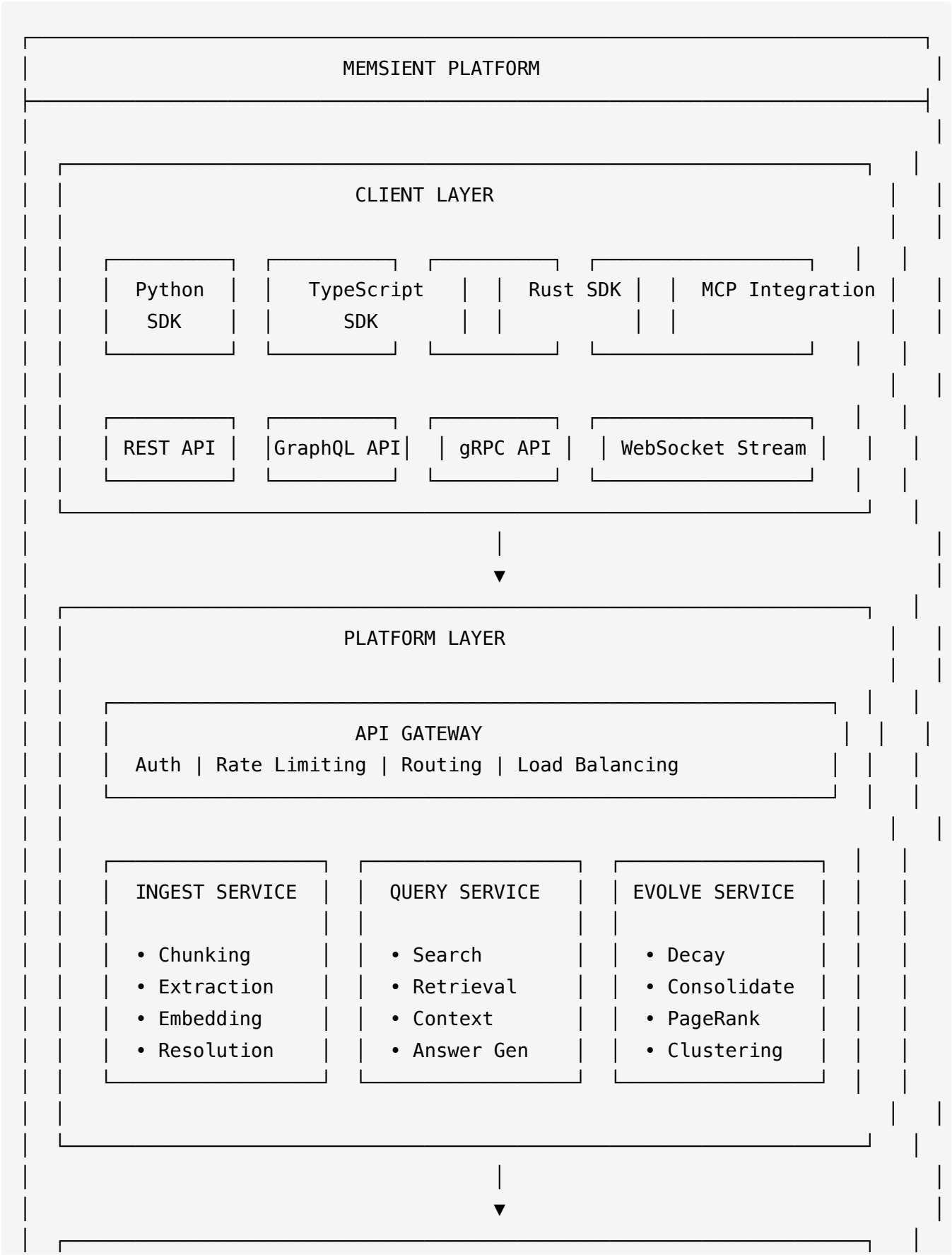
Principle 4: Privacy by Default

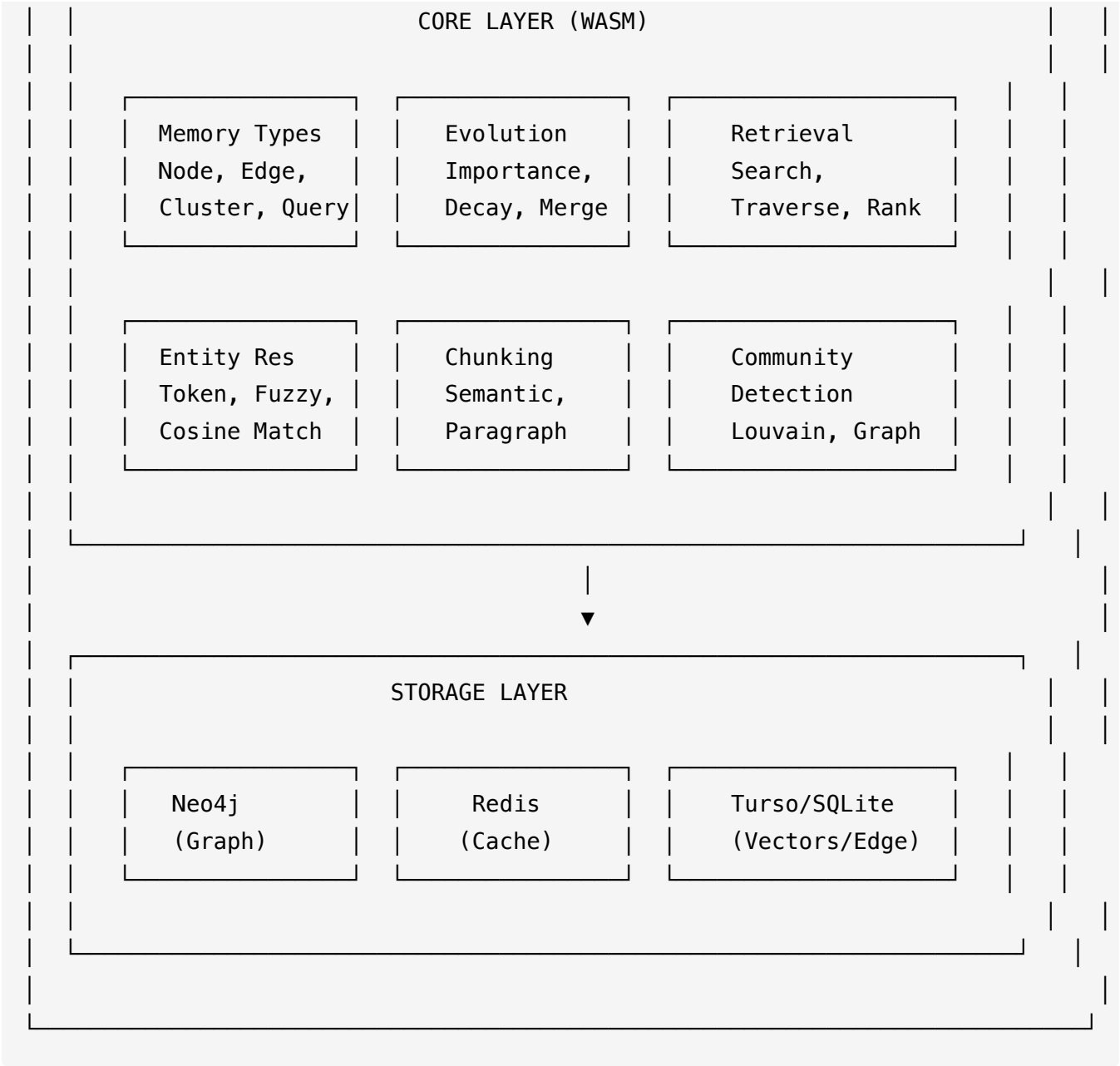
- Zero-knowledge options available
- Data never leaves customer's region
- Multi-tenant isolation guaranteed
- Right to be forgotten built-in

5.3 Product Tiers

Tier	Target	Features	Price
Developer	Individual devs	100K operations/month, community support	Free
Pro	Small teams	1M operations/month, email support	\$99/month
Team	Growing companies	10M operations/month, Slack support	\$499/month
Enterprise	Large orgs	Unlimited, SSO, SLA, dedicated support	Custom
Edge	On-device	WASM runtime, offline-first	\$999/month

5.4 Product Components





6. Core Platform Features

6.1 Feature Matrix

Category	Feature	Description	Priority
Ingestion	Document Ingestion	Ingest text, documents, conversations	P0
	Batch Ingestion	High-throughput parallel ingestion	P0
	Multimodal	Images, audio, video (future)	P2
	Streaming	Real-time ingestion via WebSocket	P1
Extraction	Entity Extraction	Automatic entity recognition	P0
	Relationship Mapping	Automatic relationship detection	P0
	Alias Resolution	Multiple names → single entity	P0
	Source Preservation	Original text chunks preserved	P0
Query	Semantic Search	Vector-based similarity search	P0
	Graph Traversal	Multi-hop relationship queries	P0
	Temporal Queries	Point-in-time queries	P0
	Natural Language	"Who was CEO in 2020?"	P0
Evolution	Memory Decay	Automatic forgetting	P0
	Consolidation	Entity/relationship merging	P0
	Importance Scoring	PageRank-based prioritization	P0
	Conflict Resolution	Newer info supersedes older	P0
Platform	Multi-Tenancy	Complete data isolation	P0

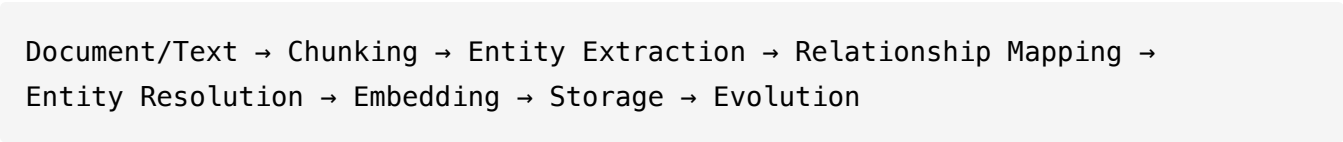
Category	Feature	Description	Priority
	RBAC	Role-based access control	P0
	Audit Logs	Complete operation history	P1
	Analytics	Usage, performance, insights	P1
Integration	MCP Server	Model Context Protocol	P0
	Python SDK	Native Python client	P0
	TypeScript SDK	Native TS/JS client	P0
	REST API	Standard HTTP API	P0
	GraphQL API	Flexible query API	P1
	Webhooks	Event notifications	P1
Enterprise	SSO/SAML	Enterprise authentication	P1
	SOC2	Security compliance	P1
	HIPAA	Healthcare compliance	P2
	On-Prem	Self-hosted deployment	P2

6.2 Feature Deep Dives

6.2.1 Memory Ingestion

What it does: Transforms raw content into structured, queryable knowledge.

Flow:



Capabilities:

- **Smart Chunking:** Respects paragraphs, sentences, code blocks
- **Exhaustive Extraction:** Every entity, relationship, number, date
- **Temporal Awareness:** Extracts "when" for every fact
- **Source Preservation:** Original text preserved for accuracy
- **Importance Tagging:** Auto-detection of critical vs routine info

Example:

```
from memsient import MemSient

ms = MemSient(api_key="ms_...")

# Ingest with automatic extraction
ms.ingest("""
    Acme Corp reported $5.2B revenue in Q4 2024, up 23% YoY.
    CEO Jane Smith announced the acquisition of TechStart Inc for $800M.
    The deal is expected to close in Q1 2025.
""")

# What gets extracted:
# ENTITIES:
#   - Acme Corp (Company)
#   - Jane Smith (Person, CEO)
#   - TechStart Inc (Company)
#   - $5.2B (Amount)
#   - $800M (Amount)
#   - Q4 2024 (Date)
#   - Q1 2025 (Date)
#
# RELATIONSHIPS:
#   - Jane Smith --[is CEO of]--> Acme Corp [valid: current]
#   - Acme Corp --[reported revenue]--> $5.2B [valid: Q4 2024]
#   - Acme Corp --[is acquiring]--> TechStart Inc [valid: Q4 2024 - Q1 2025]
#   - Acquisition --[valued at]--> $800M
```

6.2.2 Memory Query

What it does: Retrieves relevant knowledge using semantic and graph-based methods.

Query Types:

Type	Example	Method
Factual	"What is Acme's revenue?"	Semantic + Entity lookup
Relational	"Who reports to Jane?"	Graph traversal
Temporal	"Who was CEO in 2020?"	Temporal filtering
Comparative	"Compare Q3 vs Q4 revenue"	Multi-entity aggregation
Explanatory	"Why did revenue increase?"	Multi-hop reasoning

Example:

```
# Simple query
response = ms.query("What is Acme Corp's latest revenue?")
print(response.answer) # "$5.2B in Q4 2024, up 23% year-over-year"
print(response.confidence) # 0.95
print(response.sources) # [MemoryNode("Q4 2024 Earnings Report")]

# Temporal query
response = ms.query("Who was CEO of Acme in 2020?")
print(response.answer) # "John Doe was CEO from 2018 to 2022"

# Graph query
response = ms.query("What companies has Jane Smith worked at?")
print(response.answer) # "Jane Smith worked at TechGiant (2015-2020) before becoming CEO"
```

6.2.3 Memory Evolution

What it does: Automatically improves memory quality over time.

Evolution Processes:

Process	What It Does	When It Runs
Decay	Reduces strength of unused memories	Continuous

Process	What It Does	When It Runs
Consolidation	Merges duplicate entities	On ingestion + scheduled
Importance Scoring	Recalculates PageRank centrality	After graph changes
Conflict Resolution	Supersedes old info with new	On contradiction detection
Community Detection	Groups related entities	Scheduled
Summary Generation	Creates cluster summaries	After community detection

Evolution Formula:

$$\text{Importance}(e) = w1 \cdot \text{Recency}(e) + w2 \cdot \text{Frequency}(e) + w3 \cdot \text{PageRank}(e) + w4 \cdot \text{UserImportance}(e)$$

Where:

- Recency = $\exp(-0.693 \times \text{age_days} / \text{half_life})$
- Frequency = $\min(1.0, \log(1 + \text{access_count}) / \log(1 + 100))$
- PageRank = Standard damping=0.85 iteration
- UserImportance = 0.0 to 1.0 (CRITICAL=1.0, EPHEMERAL=0.0)

Example:

```
# Manual evolution
events = ms.evolve()

for event in events:
    print(f"{event.type}: {event.description}")
    # CONSOLIDATION: Merged "Acme" and "Acme Corp" into single entity
    # DECAY: Archived 15 low-importance entities from 2022
    # CONFLICT: Superseded "CEO: John Doe" with "CEO: Jane Smith"

# Automatic evolution (enabled by default)
ms = MemSient(api_key="...", auto_evolve=True)
```

6.2.4 Temporal Validity

What it does: Tracks when facts were true, not just what facts exist.

Temporal Attributes:

- `valid_from` : When the fact became true
- `valid_until` : When the fact stopped being true (null = still valid)
- `created_at` : When we learned the fact
- `updated_at` : Last modification time

Query Modes:

Mode	Query	Result
Current	"Who is CEO?"	Current CEO only
Point-in-time	"Who was CEO on Jan 1, 2020?"	CEO at that date
Range	"Who were all CEOs from 2015-2024?"	All CEOs in range
Historical	"Show CEO timeline"	Full history

Example:

```
# Current (default)
response = ms.query("Who is the CEO of Acme?")
# → "Jane Smith (since 2022)"

# Point-in-time
response = ms.query("Who was CEO of Acme on January 1, 2020?")
# → "John Doe was CEO from 2018 to 2022"

# Policy evolution
response = ms.query("What is the return policy?")
# → "60 days (updated July 2024, previously 30 days)"
```

6.2.5 Multi-Tenant Isolation

What it does: Complete data separation between customers and users.

Isolation Levels:

Level	Scope	Use Case
Organization	Company	B2B SaaS
Project	Team/App	Multiple products
User	Individual	Personalization
Session	Conversation	Chat context

Implementation:

- Separate database schemas per tenant
- Row-level security policies
- Encrypted tenant keys
- Cross-tenant queries impossible
- Audit logs per tenant

Example:

```
# Organization-level (SaaS B2B)
ms = MemSient(api_key="...", org_id="acme_corp")

# User-level (personalization)
ms = MemSient(api_key="...", org_id="acme_corp", user_id="jane@acme.com")

# Project-level (multiple agents)
ms = MemSient(api_key="...", project_id="legal_assistant")
```

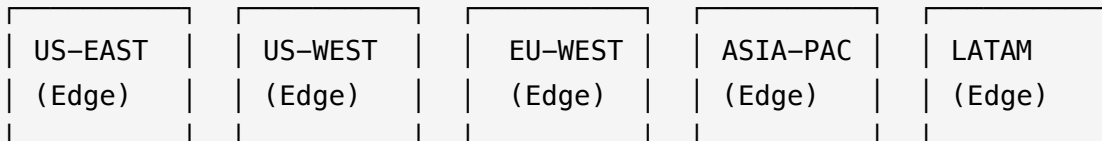


7. Technical Architecture

7.1 High-Level Architecture

MEMSIENT GLOBAL ARCHITECTURE

GLOBAL EDGE NETWORK

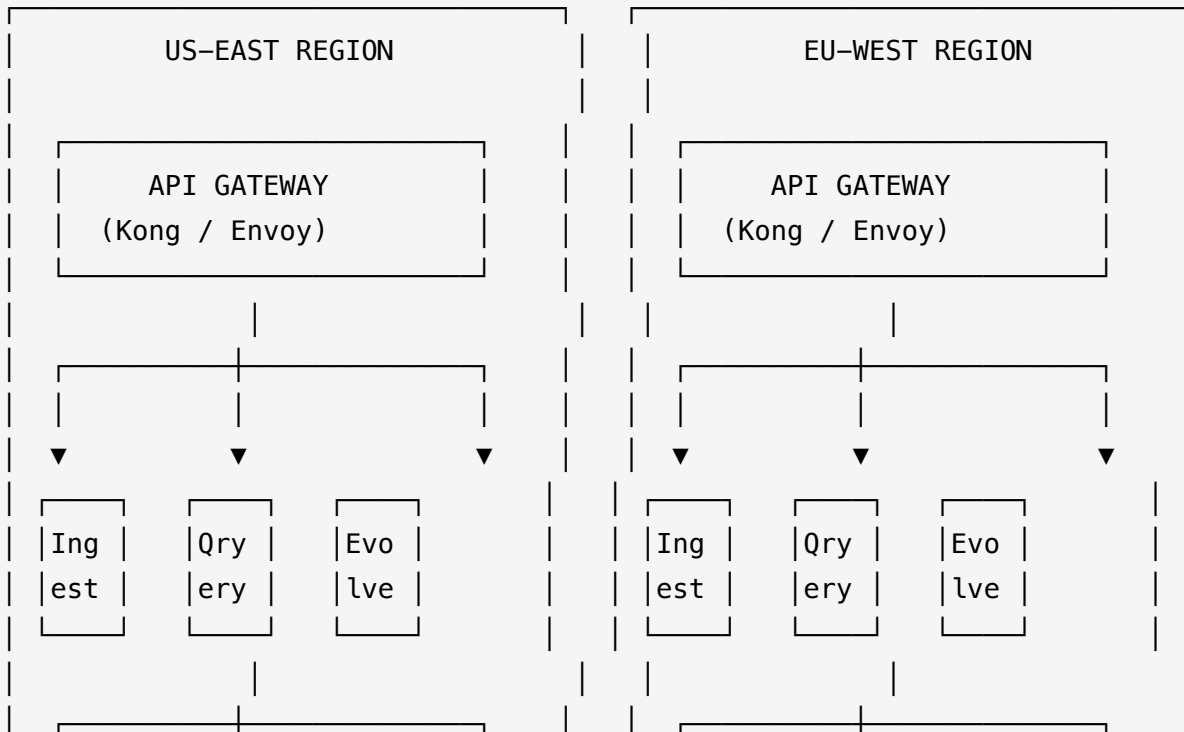


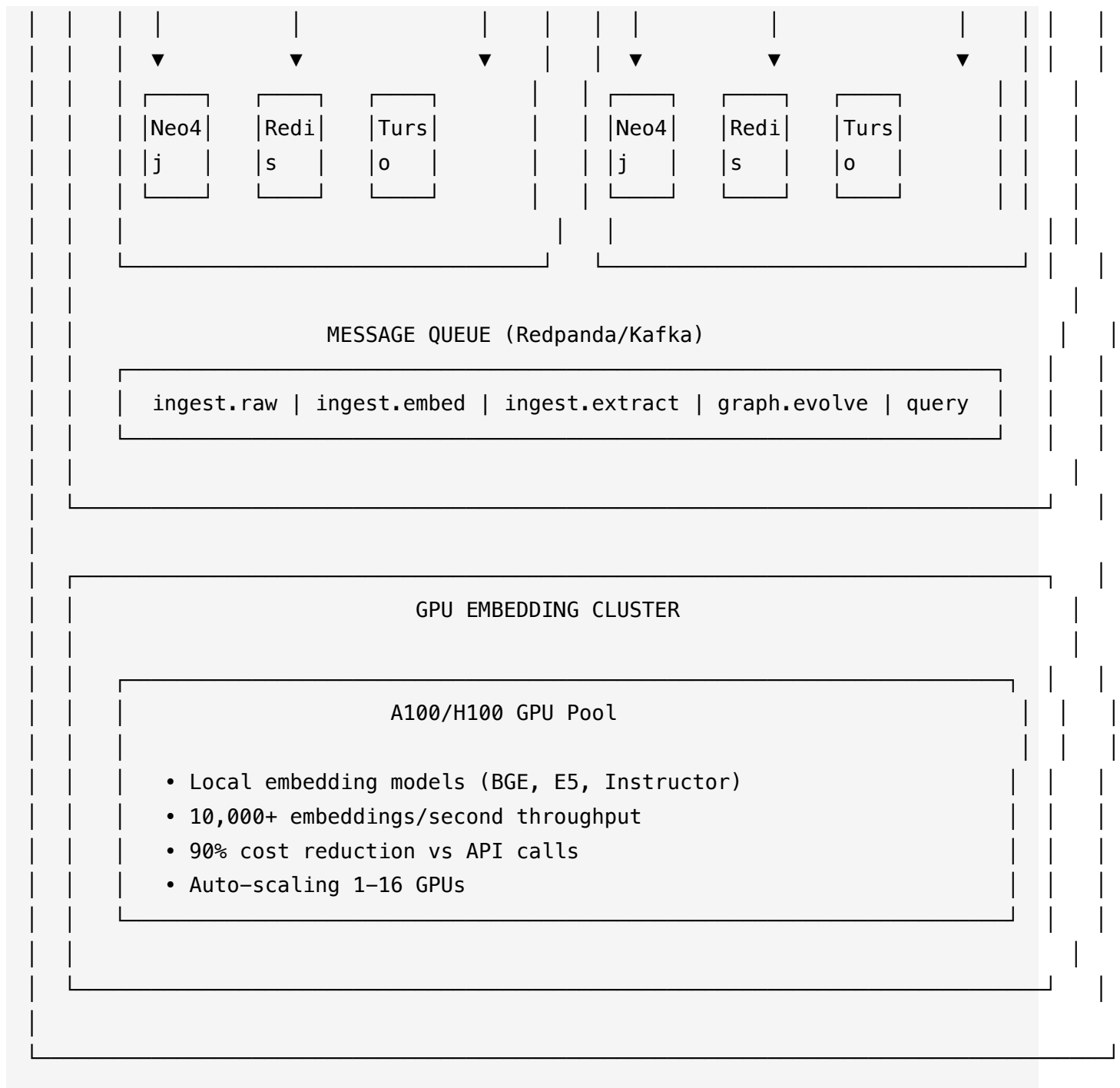
Cloudflare Workers + WASM Runtime

- Query processing < 10ms
- Local caching
- Geographic routing



REGIONAL CLUSTERS





7.2 Core Layer (Rust/WASM)

The heart of MemSient is a high-performance Rust core that compiles to:

- **Native:** Maximum performance for cloud services
- **WASM:** Edge deployment, browser, Cloudflare Workers

Core Modules

```
memsient-core/  
├─ types/           # Data structures (MemoryNode, MemoryEdge, etc.)  
├─ evolution/       # Decay, Consolidation, ImportanceScoring  
├─ graph/           # EntityResolver, CommunityDetector  
├─ retrieval/       # SemanticSearch, GraphTraversal  
├─ chunking/        # DocumentChunker  
└─ adapters/        # Trait definitions (LLM, Embedding, Storage)
```

Performance Targets

Operation	Target	Method
Entity Resolution	< 1ms	Token index + cosine
Importance Calculation	< 0.1ms	Pure math
Semantic Search	< 5ms	HNSW index
Graph Traversal (2-hop)	< 2ms	BFS with caching
Memory Decay	< 0.5ms/node	Exponential formula
Context Assembly	< 10ms	Parallel ranking

7.3 Service Layer

Ingest Service

- Document chunking
- LLM-based extraction
- Entity resolution
- Embedding generation
- Storage persistence

Query Service

- Query embedding
- Semantic search
- Graph traversal
- Context assembly
- Answer generation

Evolve Service

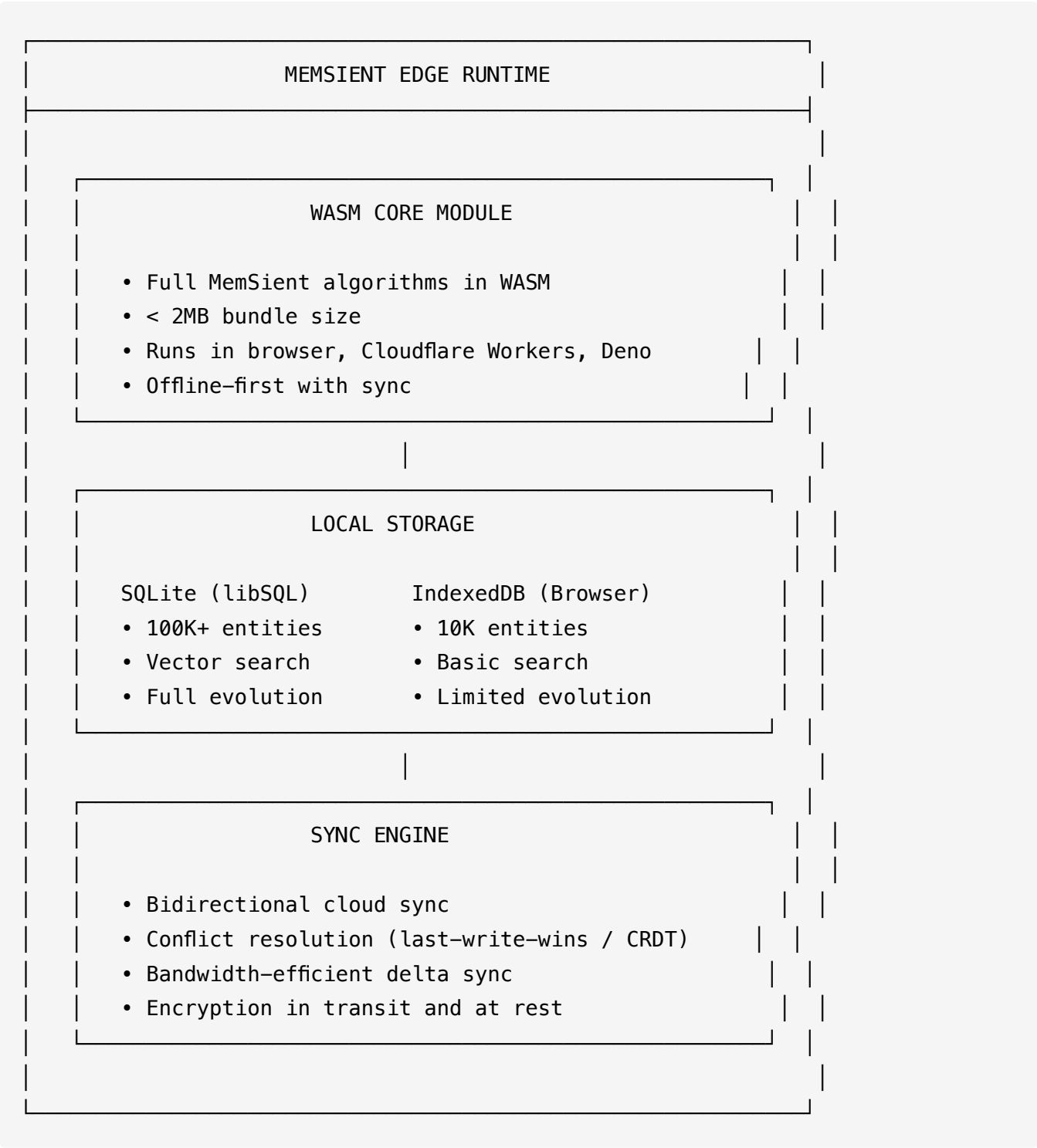
- Scheduled evolution
- Real-time consolidation
- PageRank recalculation
- Community detection
- Summary generation

7.4 Storage Layer

Component	Purpose	Technology
Graph Store	Entity/relationship storage	Neo4j Aura
Vector Store	Embedding search	Turso (libSQL)
Cache	Query/state caching	Redis Cluster
Object Store	Documents, backups	S3/R2
Message Queue	Async processing	Redpanda

7.5 Edge Architecture

For on-device and edge deployment:



Edge Deployment Targets

Platform	Runtime	Storage	Use Case
Cloudflare Workers	WASM	Durable Objects	Global edge queries

Platform	Runtime	Storage	Use Case
Browser	WASM	IndexedDB	Client-side agents
Mobile (iOS/Android)	WASM	SQLite	Mobile AI assistants
Desktop (Electron/Tauri)	Native/WASM	SQLite	IDE integrations
IoT	WASM	SQLite	Edge devices

8. Platform Components

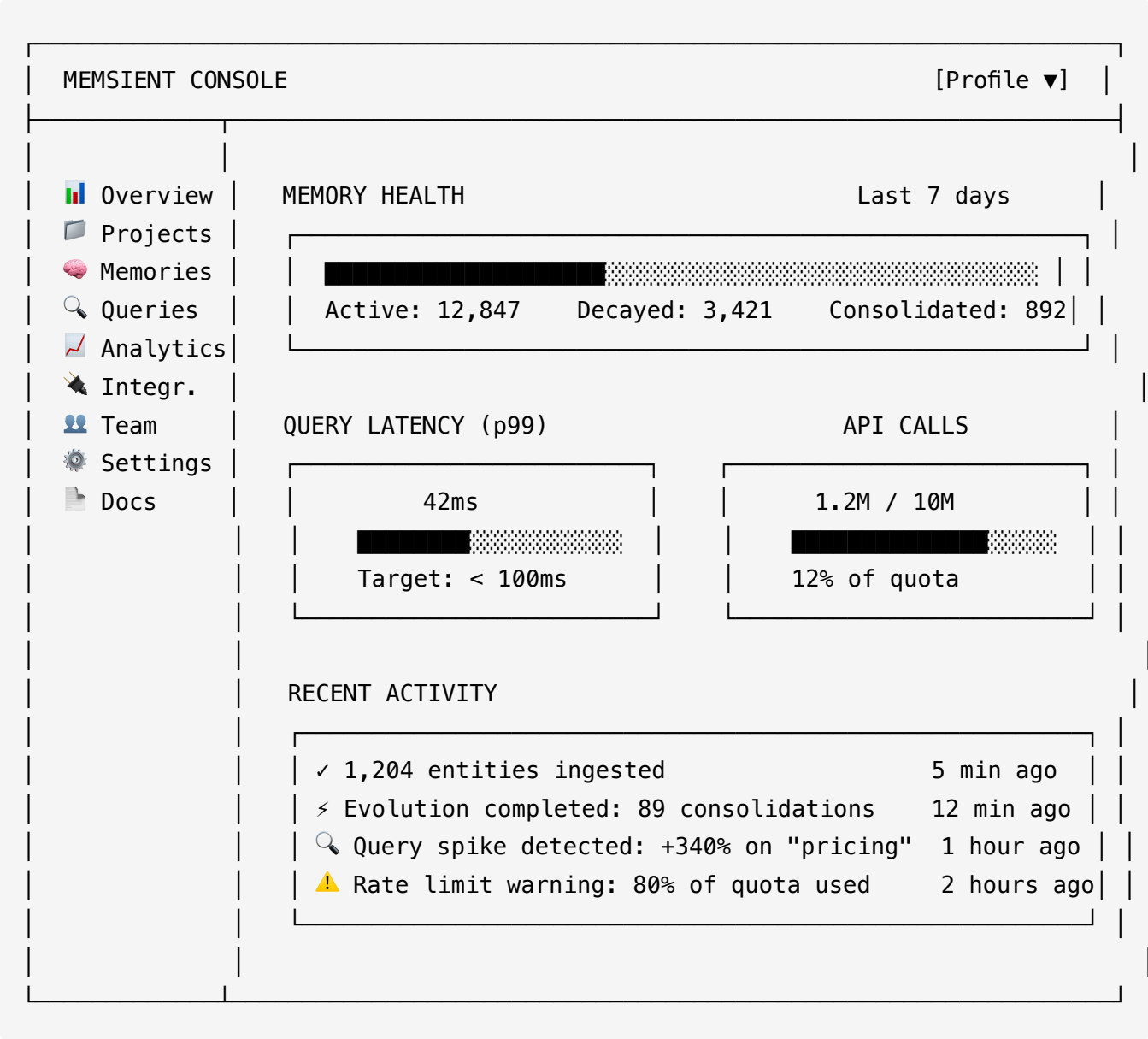
8.1 MemSient Console (Web Dashboard)

Purpose: Central management interface for organizations.

Features

Module	Functionality
Memories	Browse, search, visualize knowledge graphs
Analytics	Usage metrics, query patterns, evolution stats
Projects	Create/manage memory instances
Team	User management, RBAC, SSO configuration
Integrations	API keys, MCP setup, webhook configuration
Billing	Usage tracking, invoices, plan management
Logs	Audit logs, query history, evolution events
Playground	Interactive query testing, ingestion sandbox

Dashboard Wireframe



8.2 Knowledge Graph Visualizer

Purpose: Interactive visualization of memory structure.

Features

- **Graph View:** Force-directed layout of entities and relationships
- **Timeline View:** Temporal evolution of knowledge
- **Community View:** Clustered view of related entities

- **Search/Filter:** Find specific entities, filter by type/importance
- **Drill-down:** Click entities to see details, relationships, history
- **Export:** PNG, SVG, JSON export

8.3 Query Playground

Purpose: Test queries and ingestion without code.

Capabilities

- Natural language query input
- Raw API request builder
- Response visualization
- Latency metrics
- Context assembly viewer
- Confidence breakdown

8.4 Evolution Monitor

Purpose: Track memory health and evolution.

Metrics Displayed

Metric	Description
Decay Rate	Memories archived per day
Consolidation Rate	Entities merged per ingestion
Graph Density	Avg relationships per entity
Community Count	Number of distinct clusters
Importance Distribution	Histogram of importance scores
Temporal Coverage	Date range of valid memories

9. Integration Methods

9.1 Model Context Protocol (MCP) Server

Primary integration for AI frameworks.

MCP Configuration

```
{
  "mcpServers": {
    "memsient": {
      "command": "npx",
      "args": ["-y", "@memsient/mcp-server"],
      "env": {
        "MEMSIENT_API_KEY": "ms_...",
        "MEMSIENT_PROJECT_ID": "proj_..."
      }
    }
  }
}
```

MCP Tools Exposed

Tool	Description	Parameters
memsient_ingest	Ingest content to memory	content , metadata , importance
memsient_query	Query memory with natural language	question , temporal_context
memsient_search	Semantic search for entities	query , top_k , filters
memsient_evolve	Trigger memory evolution	force , decay_only
memsient_get_context	Get assembled context for prompt	query , max_tokens

Tool	Description	Parameters
<code>memsient_forget</code>	Explicitly forget specific memories	<code>entity_ids</code> , <code>pattern</code>

MCP Resources Exposed

Resource	Description
<code>memsient://memories</code>	List all memory instances
<code>memsient://entities/{id}</code>	Get specific entity
<code>memsient://graph/{project}</code>	Get knowledge graph summary
<code>memsient://stats</code>	Get usage statistics

Example: Cursor Integration

```
// .cursor/mcp.json
{
  "mcpServers": {
    "memsient": {
      "command": "npx",
      "args": ["-y", "@memsient/mcp-server"],
      "env": {
        "MEMSIENT_API_KEY": "${MEMSIENT_API_KEY}",
        "MEMSIENT_PROJECT_ID": "cursor_memory"
      }
    }
  }
}

// The AI assistant can now:
// 1. Remember project context across sessions
// 2. Learn user preferences over time
// 3. Recall previous debugging solutions
// 4. Track codebase architecture decisions
```

9.2 Python SDK

Installation

```
pip install memsient
# With local storage
pip install "memsient[local]"
# Full enterprise
pip install "memsient[enterprise]"
```

Basic Usage

```
from memsient import MemSient, MemoryConfig

# Initialize
ms = MemSient(
    api_key="ms_...",
    project_id="my_project",
    user_id="user_123", # Optional: user-level isolation
)

# Ingest
ms.ingest("""
    Meeting notes from Q4 planning:
    - Revenue target: $50M
    - Key initiative: Launch AI assistant
    - Owner: Sarah Chen (VP Product)
    """)

# Query
response = ms.query("What is the revenue target?")
print(response.answer)      # "$50M for Q4"
print(response.confidence)  # 0.95
print(response.sources)     # [Source(entity="Q4 Planning", ...)]

# Evolve
events = ms.evolve()
for event in events:
    print(f"{event.type}: {event.description}")

# Close (syncs if using local-first)
ms.close()
```


Advanced Usage

```
from memsient import MemSient, MemoryImportance, TemporalQuery
from datetime import datetime

# Importance levels
ms.ingest(
    "Customer allergy: peanuts – CRITICAL",
    importance=MemoryImportance.CRITICAL # Never decays
)

# Temporal queries
response = ms.query(
    "What was the revenue target?",
    temporal=TemporalQuery(as_of=datetime(2023, 1, 1))
)

# Batch ingestion
docs = [{"id": "doc1", "content": "..."}, ...]
result = ms.ingest_batch(docs, max_workers=20)

# Streaming ingestion
async for chunk in document_stream:
    await ms.ingest_async(chunk)

# Graph operations
entity = ms.get_entity("Sarah Chen")
related = ms.get_related(entity.id, relationship_type="manages")
```

9.3 TypeScript/JavaScript SDK

Installation

```
npm install @memsient/sdk
# or
yarn add @memsient/sdk
```

Usage

```
import { MemSient } from '@memsient/sdk';

// Initialize
const ms = new MemSient({
  apiKey: 'ms_...',
  projectId: 'my_project',
});

// Ingest
await ms.ingest(`
  Product launch scheduled for March 2025.
  Target markets: US, EU, APAC.
  Budget: $2M marketing.
`);

// Query
const response = await ms.query('When is the product launch?');
console.log(response.answer);      // "March 2025"
console.log(response.confidence);  // 0.92

// Streaming query
const stream = ms.queryStream('Summarize the launch plan');
for await (const chunk of stream) {
  process.stdout.write(chunk);
}

// React hook
import { useMemSient } from '@memsient/react';

function MyComponent() {
  const { query, ingest, isLoading } = useMemSient();

  const handleQuery = async () => {
    const response = await query('What is the budget?');
    setAnswer(response.answer);
  };
}
```

9.4 REST API

Endpoints

Method	Endpoint	Description
POST	/v1/ingest	Ingest content
POST	/v1/ingest/batch	Batch ingestion
POST	/v1/query	Query memory
POST	/v1/search	Semantic search
POST	/v1/evolve	Trigger evolution
GET	/v1/entities	List entities
GET	/v1/entities/{id}	Get entity
GET	/v1/graph	Get graph summary
GET	/v1/stats	Get statistics
DELETE	/v1/entities/{id}	Delete entity
DELETE	/v1/memory	Delete all memory

Authentication

```
curl -X POST https://api.memsient.com/v1/query \  
  -H "Authorization: Bearer ms_..." \  
  -H "X-Project-ID: proj_..." \  
  -H "Content-Type: application/json" \  
  -d '{"question": "Who is the CEO?"}'
```

Response Format

```
{
  "success": true,
  "data": {
    "answer": "Jane Smith has been CEO since 2022",
    "confidence": 0.94,
    "sources": [
      {
        "entity_id": "ent_abc123",
        "name": "Jane Smith",
        "type": "Person",
        "relevance": 0.98
      }
    ],
    "context_tokens": 1247,
    "latency_ms": 42
  },
  "metadata": {
    "request_id": "req_xyz789",
    "project_id": "proj_...",
    "timestamp": "2025-01-11T10:30:00Z"
  }
}
```

9.5 GraphQL API

Schema

```
type Query {
  # Query memory with natural language
  query(question: String!, temporal: TemporalInput): QueryResponse!

  # Search entities
  search(query: String!, limit: Int = 10): [Entity!]!

  # Get specific entity
  entity(id: ID!): Entity

  # Get graph structure
  graph(depth: Int = 2): Graph!

  # Get statistics
  stats: Stats!
}

type Mutation {
  # Ingest content
  ingest(content: String!, metadata: JSON, importance: Importance): IngestResult!

  # Batch ingest
  ingestBatch(documents: [DocumentInput!]!): BatchResult!

  # Trigger evolution
  evolve(force: Boolean = false): [EvolutionEvent!]!

  # Delete entity
  deleteEntity(id: ID!): Boolean!
}

type Subscription {
  # Real-time evolution events
  evolutionEvents: EvolutionEvent!

  # Ingestion progress
  ingestionProgress(batchId: ID!): Progress!
}
```

9.6 Webhooks

Events

Event	Description	Payload
<code>memory.ingested</code>	Content ingested	<code>{entities: [...], edges: [...]}</code>
<code>memory.evolved</code>	Evolution completed	<code>{events: [...]}</code>
<code>memory.conflict</code>	Conflict detected	<code>{old: {...}, new: {...}}</code>
<code>memory.decayed</code>	Entity archived	<code>{entity_id, reason}</code>
<code>query.executed</code>	Query completed	<code>{question, answer, latency}</code>
<code>quota.warning</code>	80% quota used	<code>{current, limit}</code>
<code>quota.exceeded</code>	Quota exceeded	<code>{current, limit}</code>

Configuration

```
{  
  "webhook_url": "https://your-app.com/memsient/webhook",  
  "events": ["memory.ingested", "memory.evolved", "memory.conflict"],  
  "secret": "whsec_..."  
}
```

10. Developer Experience

10.1 Documentation

Documentation Structure

`docs.memsient.com/`

└─ Getting Started/

| └─ Quick Start (5 min)

| └─ Installation

| └─ Authentication

| └─ First Memory

└─ Core Concepts/

| └─ Memory Model

| └─ Knowledge Graph

| └─ Temporal Validity

| └─ Memory Evolution

| └─ Importance Scoring

└─ Integration Guides/

| └─ MCP Setup

| └─ Python SDK

| └─ TypeScript SDK

| └─ REST API

| └─ GraphQL API

└─ Use Case Guides/

| └─ Coding Assistants

| └─ Research Agents

| └─ Customer Support

| └─ Legal AI

| └─ Financial Agents

└─ Platform Features/

| └─ Multi-Tenancy

| └─ Edge Deployment

| └─ Enterprise SSO

| └─ Compliance

└─ API Reference/

| └─ Python

| └─ TypeScript

| └─ REST

| └─ GraphQL

└─ Resources/

└─ Changelog

└─ Migration Guides

└─ Best Practices

└─ FAQ

10.2 CLI Tool

```
# Install
npm install -g @memsient/cli
# or
pip install memsient-cli

# Login
memsient login

# Project management
memsient projects list
memsient projects create my-project
memsient projects switch my-project

# Quick ingestion
memsient ingest "Important information..."
memsient ingest --file document.txt
memsient ingest --dir ./documents --recursive

# Query
memsient query "What is the budget?"

# Stats
memsient stats

# Evolution
memsient evolve

# Export/Import
memsient export --format json > backup.json
memsient import backup.json
```

10.3 Local Development

Docker Compose for Local Dev

```
# docker-compose.yml
version: '3.8'
services:
  memsient:
    image: memsient/dev:latest
    ports:
      - "8080:8080"      # API
      - "8081:8081"      # Console
      - "9090:9090"      # Metrics
    environment:
      - MEMSIENT_DEV_MODE=true
      - MEMSIENT_LLM_PROVIDER=ollama
      - OLLAMA_HOST=http://ollama:11434
    volumes:
      - ./data:/data
    depends_on:
      - ollama

  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama-data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]

volumes:
  ollama-data:
```

10.4 Developer Portal

Features

Feature	Description
API Keys	Generate, rotate, revoke keys
Usage Dashboard	Real-time API usage
Request Inspector	Debug failed requests
Webhook Logs	Test and debug webhooks
Code Examples	Copy-paste examples in 5+ languages
Changelog	API changes and deprecations
Status Page	System health and incidents

11. Infrastructure & Deployment

11.1 Cloud Regions

Initial Launch (Phase 1)

Region	Location	Services
us-east-1	Virginia, US	Full stack
eu-west-1	Ireland, EU	Full stack (GDPR)

Expansion (Phase 2)

Region	Location	Services
us-west-2	Oregon, US	Full stack
ap-northeast-1	Tokyo, Japan	Full stack
ap-southeast-1	Singapore	Full stack

Edge Presence (Global)

Cloudflare Workers in 300+ edge locations for:

- Query caching (< 10ms)
- Geographic routing
- DDoS protection
- Rate limiting

11.2 Infrastructure Stack

Component	Technology	Justification
Compute	Kubernetes (GKE/EKS)	Auto-scaling, multi-cloud
API Gateway	Kong / Envoy	Rate limiting, auth, routing
Graph DB	Neo4j Aura	Native graph, ACID, managed
Vector DB	Turso (libSQL)	Edge-compatible, vectors
Cache	Redis Cluster	Sub-ms latency, pub/sub
Queue	Redpanda	Kafka-compatible, low latency
Object Storage	S3 / R2	Documents, backups
CDN/Edge	Cloudflare	Global edge, WASM workers
GPU	A100/H100 (GCP/AWS)	Embedding generation

Component	Technology	Justification
Monitoring	Prometheus + Grafana	Metrics, alerting
Logging	Loki	Log aggregation
Tracing	Jaeger	Distributed tracing

11.3 Scaling Strategy

Horizontal Scaling

Service	Scaling Trigger	Min	Max
API Gateway	CPU > 70%	3	50
Ingest Workers	Queue lag > 1000	5	100
Query Workers	CPU > 60%	10	100
Evolve Workers	Scheduled	2	20
Embedding Workers	Queue lag > 500	2	16

Capacity Targets

Metric	Target
Ingestion	10,000 docs/second
Queries	100,000 queries/second
Entities	1 billion+
Concurrent Users	100,000+

11.4 Disaster Recovery

Scenario	RTO	RPO	Strategy
Single node failure	0	0	Auto-failover
AZ failure	< 1 min	0	Multi-AZ deployment
Region failure	< 15 min	< 1 min	Cross-region replication
Data corruption	< 1 hour	< 5 min	Point-in-time recovery

12. Security, Privacy & Compliance

12.1 Security Architecture

SECURITY ARCHITECTURE

PERIMETER SECURITY

- WAF (Cloudflare)
- Rate Limiting
- IP Allowlisting (Enterprise)
- DDoS Protection
- Bot Detection

ACCESS CONTROL

- API Key Authentication
- OAuth 2.0 / OIDC
- Role-Based Access Control
- JWT Tokens
- SAML SSO (Enterprise)
- Fine-grained Permissions

DATA SECURITY

- Encryption at Rest (AES-256-GCM)
- Encryption in Transit (TLS 1.3)
- Customer-Managed Keys (Enterprise)
- Field-Level Encryption

TENANT ISOLATION

- Logical isolation (row-level security)
- Physical isolation (dedicated clusters – Enterprise)
- Network isolation (VPC peering)
- Cross-tenant access impossible

12.2 Compliance Roadmap

Certification	Target Date	Status
SOC 2 Type I	Q2 2025	 In Progress
SOC 2 Type II	Q4 2025	 Planned
GDPR	Launch	 Ready
CCPA	Launch	 Ready
HIPAA	Q3 2025	 Planned
ISO 27001	Q1 2026	 Planned
FedRAMP	2026	 Planned

12.3 Privacy Features

Feature	Description
Data Residency	Choose region (US, EU, APAC)
Right to Erasure	Complete data deletion API
Data Portability	Full export in standard formats
Consent Management	Track consent for data usage
Audit Logs	Complete operation history
Anonymization	PII detection and masking

12.4 Enterprise Security

Feature	Description
SSO/SAML	Okta, Azure AD, Google Workspace

Feature	Description
SCIM	Automated user provisioning
IP Allowlisting	Restrict API access by IP
VPC Peering	Private network connectivity
Dedicated Clusters	Physical isolation
Custom Encryption Keys	BYOK support
Security Questionnaires	Pre-filled responses

13. Business Model & Pricing

13.1 Pricing Philosophy

1. **Value-Based:** Price correlates with value delivered
2. **Predictable:** No surprise bills
3. **Scalable:** Grows with customer
4. **Developer-Friendly:** Generous free tier

13.2 Pricing Tiers

Developer (Free)

Feature	Limit
Operations/month	100,000
Storage	1 GB
Projects	3

Feature	Limit
Support	Community
SLA	None

Target: Individual developers, learning, prototypes

Pro (\$99/month)

Feature	Limit
Operations/month	1,000,000
Storage	10 GB
Projects	10
Team Members	5
Support	Email (48h)
SLA	99.9%

Target: Small teams, startups, MVPs

Team (\$499/month)

Feature	Limit
Operations/month	10,000,000
Storage	100 GB
Projects	Unlimited
Team Members	25
Support	Slack (24h)
SLA	99.95%

Feature	Limit
Analytics	Advanced
RBAC	Full

Target: Growing companies, production workloads

Enterprise (Custom)

Feature	Limit
Operations/month	Unlimited
Storage	Unlimited
Projects	Unlimited
Team Members	Unlimited
Support	Dedicated CSM + 4h SLA
SLA	99.99%
SSO/SAML	✓
Custom Contracts	✓
On-Premises	Optional
Dedicated Clusters	Optional

Target: Large organizations, regulated industries

Edge Runtime (\$999/month)

Feature	Limit
Edge Deployments	Unlimited
Local Storage	Unlimited

Feature	Limit
Cloud Sync	10M ops/month
Support	Email (24h)

Target: On-device AI, offline-first applications

13.3 Usage-Based Pricing (Overage)

Operation Type	Pro	Team	Enterprise
Additional 1M operations	\$50	\$40	Custom
Additional 10 GB storage	\$20	\$15	Custom
Additional team member	\$10	\$8	Included

13.4 Revenue Projections

Year	Customers	ARR	Revenue Mix
Y1	500	\$10M	60% Enterprise, 25% Team, 15% Pro
Y2	2,000	\$35M	55% Enterprise, 30% Team, 15% Pro
Y3	5,000	\$100M	50% Enterprise, 35% Team, 15% Pro

14. Go-to-Market Strategy

14.1 Launch Phases

Phase 1: Private Beta (Q1 2025)

Goal: Validate product-market fit with design partners

- **Partners:** 20 hand-selected companies
- **Focus:** Coding assistants, research agents
- **Pricing:** Free
- **Success Metric:** 80% want to continue

Phase 2: Public Beta (Q2 2025)

Goal: Scale to 500+ developers

- **Access:** Open waitlist
- **Pricing:** Free tier only
- **Focus:** Developer experience, documentation
- **Success Metric:** 10,000 signups, 500 active

Phase 3: General Availability (Q3 2025)

Goal: Launch paid tiers

- **Access:** Open signup
- **Pricing:** Full tier structure
- **Focus:** Enterprise features, compliance
- **Success Metric:** \$1M ARR by end of Q3

Phase 4: Enterprise Push (Q4 2025)

Goal: Land enterprise contracts

- **Focus:** SOC 2, HIPAA, SSO

- **Sales:** Outbound + inbound
- **Success Metric:** \$5M ARR by end of Q4

14.2 Marketing Channels

Developer Marketing

Channel	Tactic	Budget
Content	Blog, tutorials, case studies	\$10K/mo
Social	Twitter/X, LinkedIn, Reddit	\$5K/mo
Community	Discord, GitHub, Slack	\$5K/mo
Conferences	AI/ML conferences, sponsor booths	\$20K/quarter
Influencers	AI YouTubers, newsletter sponsors	\$10K/mo
SEO	Documentation, landing pages	\$5K/mo

Enterprise Marketing

Channel	Tactic	Budget
Sales	Outbound SDRs, AEs	\$50K/mo
Events	Enterprise AI summits	\$30K/quarter
Content	Whitepapers, webinars	\$10K/mo
Partnerships	SI relationships	\$20K/quarter
ABM	Account-based campaigns	\$20K/mo

14.3 Partnership Strategy

Technology Partnerships

Partner Type	Examples	Integration
AI Frameworks	LangChain, LlamaIndex, CrewAI	Native integration
LLM Providers	OpenAI, Anthropic, Google	Embedding/LLM support
Cloud Providers	AWS, GCP, Azure	Marketplace listing
AI Agents	Cursor, Windsurf, Cody	Pre-built integration

Channel Partnerships

Partner Type	Examples	Model
System Integrators	Accenture, Deloitte	Implementation partners
Resellers	Regional AI consultancies	Revenue share
OEM	Embedded in partner products	White-label licensing

14.4 Content Strategy

Content Pillars

- Memory Science:** How cognitive memory applies to AI
- Agent Building:** Practical guides for developers
- Enterprise AI:** Case studies, ROI analysis
- Technical Deep Dives:** Architecture, benchmarks
- Vision:** Path to AGI, AI safety

Content Calendar

Week	Blog	Tutorial	Case Study	Webinar
W1	Vision post	Quick start	-	-
W2	Memory decay explained	MCP setup	Coding assistant	-
W3	PageRank for AI	Python SDK	-	Live demo
W4	Enterprise security	Edge deployment	Legal AI	-

15. Success Metrics & KPIs

15.1 North Star Metric

Total Memory Operations Processed

This metric captures:

- Product usage (adoption)
- Customer value (more ops = more value)
- Revenue correlation (usage-based pricing)

Targets:

- Y1: 10 billion ops/month
- Y2: 100 billion ops/month
- Y3: 1 trillion ops/month

15.2 Product Metrics

Adoption

Metric	Q1	Q2	Q3	Q4
Signups	2K	10K	25K	50K
Active Projects	500	2K	5K	10K
Daily Active Users	100	500	2K	5K
MCP Integrations	50	200	500	1K

Engagement

Metric	Target
Queries/Project/Day	> 100
Ingestions/Project/Week	> 50
Evolution Cycles/Project/Week	> 3
API Calls/Active User/Day	> 50

Quality

Metric	Target
Query Confidence (avg)	> 0.85
Ingestion Success Rate	> 99.5%
Entity Resolution Accuracy	> 95%
Conflict Resolution Correct	> 90%

15.3 Technical Metrics

Metric	Target
Query Latency (p50)	< 50ms
Query Latency (p99)	< 100ms
Ingestion Throughput	> 10K docs/sec
API Uptime	> 99.99%
Error Rate	< 0.1%

15.4 Business Metrics

Metric	Q1	Q2	Q3	Q4
MRR	\$50K	\$200K	\$500K	\$1M
ARR	\$600K	\$2.4M	\$6M	\$12M
Paying Customers	20	100	300	500
Enterprise Customers	2	10	25	50
ACV (Enterprise)	\$100K	\$100K	\$120K	\$150K

Unit Economics

Metric	Target
CAC (Payback)	< 12 months
LTV:CAC	> 3:1
Net Revenue Retention	> 120%
Gross Margin	> 70%

Metric	Target
Logo Churn	< 5% annually

16. Product Roadmap

16.1 2025 Roadmap

Q1 2025: Foundation

Feature	Priority	Status
Core Platform (Cloud)	P0	
Python SDK	P0	
TypeScript SDK	P0	
REST API	P0	
MCP Server	P0	
Web Console (Basic)	P0	
Documentation Site	P0	
Private Beta Launch	P0	

Q2 2025: Scale

Feature	Priority	Status
Public Beta Launch	P0	
GraphQL API	P1	

Feature	Priority	Status
Advanced Analytics	P1	
Team Management	P0	
Webhooks	P1	
CLI Tool	P1	
Rate Limiting Dashboard	P1	
Knowledge Graph Visualizer	P1	

Q3 2025: Enterprise

Feature	Priority	Status
General Availability	P0	
SSO/SAML	P1	
SOC 2 Type I	P1	
Audit Logs	P1	
Custom Domains	P2	
VPC Peering	P2	
gRPC API	P2	
Go SDK	P2	

Q4 2025: Edge

Feature	Priority	Status
WASM Edge Runtime	P1	
Browser SDK	P1	

Feature	Priority	Status
Cloudflare Workers Integration	P1	
Offline-First Mode	P1	
SOC 2 Type II	P1	
HIPAA Compliance	P2	
Mobile SDKs (iOS/Android)	P2	

16.2 2026+ Vision

Q1-Q2 2026: Intelligence

- **Skill Acquisition Engine:** Learn and apply reusable skills
- **Program Abstraction:** Extract and generalize patterns
- **Cross-Memory Learning:** Learn from aggregated (anonymized) patterns

Q3-Q4 2026: Collective

- **Multi-Agent Memory Sharing:** Agents share knowledge
- **Federated Learning:** Privacy-preserving collective intelligence
- **Memory Marketplace:** Buy/sell domain knowledge graphs

2027+: AGI Path

- **Meta-Cognition Layer:** Memory that reflects on itself
 - **World Modeling:** Build and maintain world models
 - **Reasoning Integration:** Deep integration with reasoning systems
 - **ARC-AGI Benchmark:** Target 100% score
-

17. Competitive Analysis

17.1 Competitive Matrix

Feature	MemSient	Pinecone	Weaviate	Mem0	LangChain Memory
Memory Decay	✓	✗	✗	✗	✗
Consolidation	✓	✗	✗	✗	✗
Temporal Validity	✓ Full	✗	Basic	✗	✗
Knowledge Graph	✓ Native	✗	✓	✗	✗
PageRank Importance	✓	✗	✗	✗	✗
Conflict Resolution	✓ Auto	✗	✗	✗	✗
Self-Evolution	✓	✗	✗	✗	✗
MCP Native	✓	✗	✗	✗	✗
Edge/WASM	✓	✗	✗	✗	✗
Multi-Tenant	✓	✓	✓	✗	✗
Enterprise Ready	✓	✓	✓	✗	✗
Pricing	Usage	Usage	Usage	Usage	Free (OSS)

17.2 Competitive Positioning

vs. Pinecone

Aspect	Pinecone	MemSient
Focus	Vector search	Cognitive memory
Strength	Scale, reliability	Intelligence, evolution
Weakness	Just storage	Newer, less proven
Best For	Static RAG	Dynamic agents

MemSient Advantage: Not just vectors — full knowledge graph with temporal awareness and self-evolution.

vs. Weaviate

Aspect	Weaviate	MemSient
Focus	Vector + graph	Cognitive memory
Strength	GraphQL, flexibility	Memory science
Weakness	No evolution	Newer
Best For	Custom search	Agent memory

MemSient Advantage: Purpose-built for AI agents with decay, consolidation, and conflict resolution.

vs. Mem0

Aspect	Mem0	MemSient
Focus	Simple memory API	Full memory platform
Strength	Easy to use	Feature-complete

Aspect	Mem0	MemSient
Weakness	Limited features	Complexity
Best For	Quick prototypes	Production agents

MemSient Advantage: Enterprise-grade with temporal queries, multi-tenancy, and edge deployment.

vs. Custom Solutions

Aspect	Custom	MemSient
Focus	Company-specific	Universal
Strength	Full control	Time to market
Weakness	6-12 month build	Platform dependency
Best For	Unique requirements	Standard agents

MemSient Advantage: 6 months of development in a single API call. Focus on your agent, not infrastructure.

17.3 Defensibility

Technical Moats

- 1. **Rust/WASM Core:** 10x faster than Python alternatives
- 2. **Cognitive Algorithms:** Years of neuroscience research
- 3. **Edge Runtime:** Unique WASM-based edge deployment
- 4. **MCP Native:** First-mover on Model Context Protocol

Business Moats

- 1. **Developer Community:** Strong open-source foundation
- 2. **Enterprise Relationships:** SOC 2, HIPAA, dedicated support
- 3. **Data Effects:** More usage → better evolution algorithms

4. **Switching Costs:** Deep integration into agent workflows

18. Risk Assessment & Mitigation

18.1 Technical Risks

Risk	Probability	Impact	Mitigation
Performance at Scale	Medium	High	Horizontal scaling, load testing
LLM API Reliability	Medium	High	Multi-provider fallback, caching
Data Loss	Low	Critical	Multi-region replication, backups
Security Breach	Low	Critical	SOC 2, penetration testing, encryption
Edge Runtime Bugs	Medium	Medium	Extensive WASM testing, gradual rollout

18.2 Business Risks

Risk	Probability	Impact	Mitigation
Slow Adoption	Medium	High	Aggressive free tier, developer marketing
Enterprise Sales Cycle	High	Medium	Focus on mid-market initially
Competitor Response	High	Medium	Move fast, build moats
Pricing Pressure	Medium	Medium	Value-based pricing, differentiation

Risk	Probability	Impact	Mitigation
Key Person Risk	Medium	Medium	Knowledge sharing, documentation

18.3 Market Risks

Risk	Probability	Impact	Mitigation
AI Winter	Low	High	Diversify to non-AI use cases
LLM Context Expansion	Medium	Medium	Focus on features beyond context
Regulatory Changes	Medium	Medium	Compliance-first approach
Economic Downturn	Medium	Medium	Efficient operations, runway

19. Team & Resources

19.1 Current Team

Role	Name	Experience
Founder/CEO	Al-Amin Ibrahim	AI/ML, Enterprise Software
Technical Advisors	Multiple	AI Research, Systems

19.2 Key Hires (Year 1)

Role	Priority	Timeline	Responsibility
CTO	P0	Q1	Technical vision, architecture

Role	Priority	Timeline	Responsibility
Head of Engineering	P0	Q1	Team building, delivery
Senior Backend Engineer (Rust)	P0	Q1-Q2	Core platform
Senior Frontend Engineer	P0	Q1-Q2	Console, SDKs
DevRel Lead	P0	Q2	Community, content
Head of Sales	P1	Q2-Q3	Enterprise sales
Solutions Engineer	P1	Q3	Enterprise support
Security Engineer	P1	Q2	Compliance, security

19.3 Resource Needs

Engineering

Resource	Year 1	Year 2
Headcount	10	25
Cost	\$2M	\$5M

Infrastructure

Resource	Year 1	Year 2
Cloud	\$500K	\$2M
SaaS Tools	\$100K	\$300K

Go-to-Market

Resource	Year 1	Year 2
Marketing	\$500K	\$1.5M
Sales	\$300K	\$1M

Total Budget

Category	Year 1	Year 2
Total	\$3.4M	\$9.8M

20. Appendices

Appendix A: Glossary

Term	Definition
Memory Node	An entity in the knowledge graph (person, company, concept)
Memory Edge	A relationship between nodes
Memory Decay	Automatic reduction of importance over time
Consolidation	Merging of duplicate or related entities
Temporal Validity	Time range when a fact was/is true
Importance Score	Weighted score determining decay resistance
PageRank	Graph centrality algorithm for importance
MCP	Model Context Protocol (Anthropic standard)
Evolution	Automatic memory improvement process

Term	Definition
Community	Cluster of related entities

Appendix B: API Rate Limits

Tier	Requests/Min	Burst	Concurrent
Developer	60	10	5
Pro	300	50	20
Team	1000	100	50
Enterprise	Custom	Custom	Custom

Appendix C: Data Retention

Tier	Active Data	Archived Data	Backups
Developer	30 days	7 days	None
Pro	1 year	90 days	Weekly
Team	Unlimited	1 year	Daily
Enterprise	Unlimited	Unlimited	Continuous

Appendix D: SLA Details

Tier	Uptime	Credits
Pro	99.9%	10% for < 99.5%
Team	99.95%	10% for < 99.9%, 25% for < 99.5%
Enterprise	99.99%	Custom

Appendix E: Research Foundation

Academic References

1. Ebbinghaus, H. (1885). *Memory: A Contribution to Experimental Psychology*
2. McClelland, J. et al. (1995). *Why There Are Complementary Learning Systems*
3. Page, L. & Brin, S. (1998). *The PageRank Citation Ranking*
4. Blondel, V. et al. (2008). *Fast Unfolding of Communities in Large Networks*
5. Vaswani, A. et al. (2017). *Attention Is All You Need*

MemSient Research Paper

- **Title:** GraphMem: A Self-Evolving Graph-Based Memory Architecture for AI Agents
- **Status:** Published on ResearchGate
- **Link:** <https://www.researchgate.net/publication/398203328>

Document Control

Version	Date	Author	Changes
1.0	2025-01-11	Al-Amin Ibrahim	Initial PRD



MemSient

The Sentient Memory Platform for AI Agents

"Memory is not storage — it's the foundation of intelligence."